BIZNAGA
FEST

BIZNAGAFEST 2026 · PLAYER 1

Construyendo la Web Agéntica

Introducción, Utilidades y Retos de Web-MCP

Raúl Tovar

Senior Software Engineer · The Workshop

07.11.2026 · 12:00

Player 1

- Raúl Tovar, Senior Software Engineer en **The Workshop**
- Backend y frontend: me interesa todo lo que pasa entre la base de datos y el navegador
- Laboratorio de esta charla: **knot.kz**, un side-project donde pruebo WebMCP en producción

La web está hecha para humanos

- Los agentes usan la interfaz pensada para ti: la leen a base de **capturas de pantalla** y DOM
- Frágil: el CSS puede despistar al agente
- Caro: miles de tokens para "leer" una página
- Ciego: no sabe qué hace un botón hasta que lo pulsa

> ¿Y si la página pudiera decirle al agente qué sabe hacer?

ROUND 1

Introducción

De MCP a Web-MCP

MCP en 60 segundos

- **Model Context Protocol**: estándar abierto para conectar LLMs con herramientas
- Arquitectura host → client → server
- El servidor expone **tools**, resources y prompts
- Se comunica por stdio (proceso local) o HTTP (servicio remoto)
- Spec actual **2026-07-28**: **sin estado**, cada petición se describe a sí misma

Web-MCP: la página es el servidor

- Web-MCP es una propuesta de API web, inspirada en MCP, que aún no es estándar
- Draft del W3C Web Machine Learning Community Group (Microsoft + Google)
- La web publica sus tools **desde el frontend**, con JavaScript o con HTML
- El navegador media cada llamada: la tool corre en la página, **con la sesión del usuario**

> "Instead of your application being a guest within an agent, the agent is a guest on your platform."

¿Dónde funciona hoy?

NAVEGADOR / AGENTE	ESTADO
Chrome	Origin trial desde **149** · flag `#enable-webmcp-testing`
Edge	Origin trial desde **150**
Brave	Experimental en Leo AI · flag `#enable-webmcp-testing`
Firefox	Postura **neutral** · sin implementar
Safari	**En contra**
ChatGPT (escritorio)	**Site tools** en su navegador integrado (agosto 2026)

MCP vs Web-MCP

	MCP	WEB-MCP
Dónde vive	Fuera del navegador	Dentro de la pestaña
Duración	Mientras el servidor MCP esté en marcha	**Mientras la pestaña esté abierta**
Configuración	Requiere configuración en tu cliente	Basta con abrir la página
El usuario ve	Solo la respuesta del agente	La página cambiando en directo

> "WebMCP is not an extension or a replacement of MCP."

ROUND 2

Utilidades

Dos APIs, un mismo objetivo

API imperativa

```
// 1. Lo que el agente puede mandar
const inputSchema = { type: 'object', properties: { limit: { type: 'number', minimum: 1, maximum: 4 } } }

// 2. Lo que se ejecuta en la página
async function execute(input) {
  const data = await releases(input.limit ?? 3)   // código que la web ya tenía
  return { content: [{ type: 'text', text: JSON.stringify(data) }] }
}

// 3. La tool completa
const tool = {
  name: 'get-latest-releases',
  description: 'Returns the latest releases of Knot and Knot Server...',
  annotations: { readOnlyHint: true, untrustedContentHint: true },
  inputSchema, execute,
}

// 4. Registrarla; controller.abort() la quita
const controller = new AbortController()
await document.modelContext.registerTool(tool, { signal: controller.signal })
```

API declarativa: formularios en segundos

```
<form toolname="contact-knot-team"
      tooldescription="Send a message to the Knot maintainers..."
      action="/api/contact" method="post">
  <input name="email" type="email" required
         toolparamdescription="Reply-to address of the person contacting the team">
  <select name="topic" toolparamdescription="Category: support | bug | other">…</select>
  <textarea name="message" required toolparamdescription="Body of the message"></textarea>
  <button type="submit">Send message</button>
</form>
```

- Sin JavaScript obligatorio: el navegador **genera el JSON Schema** desde el formulario
- Sin `toolautosubmit`: el agente rellena, **el humano pulsa Send**

¿Y mi framework?

FRAMEWORK	SOPORTE
Angular	**Oficial**, experimental desde **v22** · Signal Forms → tool
React	Sin soporte oficial · `use-webmcp-tool` (Google Chrome Labs, experimental)
Vue / Nuxt	Comunidad: `vue-webmcp`, `nuxt-webmcp`
Cualquiera	Polyfill y tipos: `@mcp-b/global`, `webmcp-types`

knot.kz: **sin librerías**, un hook de ~70 líneas. Una librería no te ahorra código: te aísla de los **cambios de la spec**.

Anotaciones

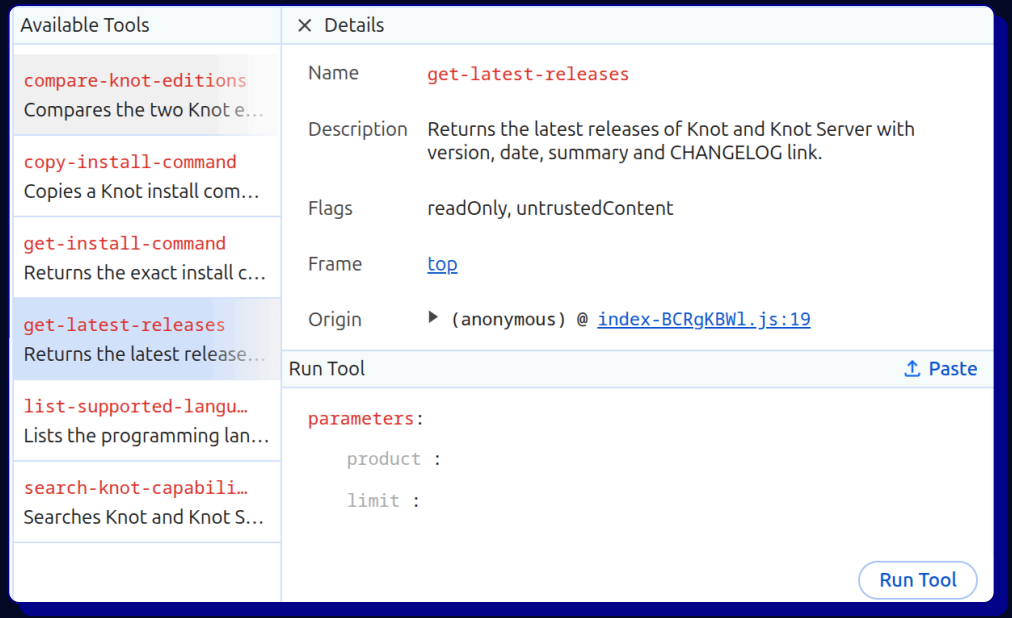
- `readOnlyHint`: la tool no modifica estado
- `untrustedContentHint`: lo que devuelve puede contener texto hostil
- `consequentialHint`: acción irreversible → el agente debería pedir confirmación
- Todas valen `false` por defecto

```
// getTools() en knot.kz · Chrome 155
{
  name: 'get-latest-releases',
  origin: 'https://www.knot.kz',
  annotations: {
    consequentialHint: false,
    readOnlyHint: true,
    untrustedContentHint: true,
  },
}
```

Herramientas

- **Desarrollo**: flag · **Usuarios**: origin trial
- **DevTools**: listar, inspeccionar y ejecutar tools
- Model Context Tool Inspector: chatea con tus tools
- **chrome-devtools-mcp**, **Playwright MCP** y **chrome-debug**: conectan cualquier agente MCP con las tools de una pestaña

Pruébalo: knot.kz → F12 → Application → WebMCP



BONUS STAGE

Demo

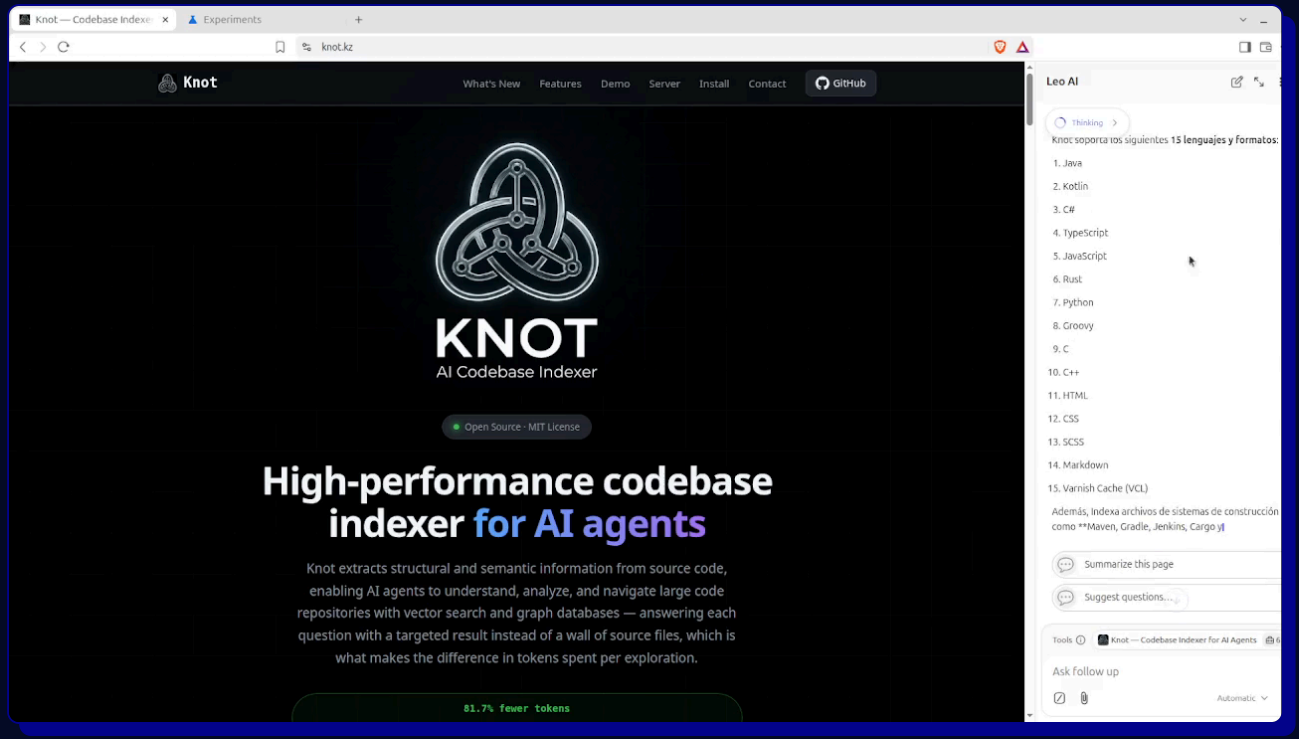
Player 2 has entered the game

Lo que vamos a ver

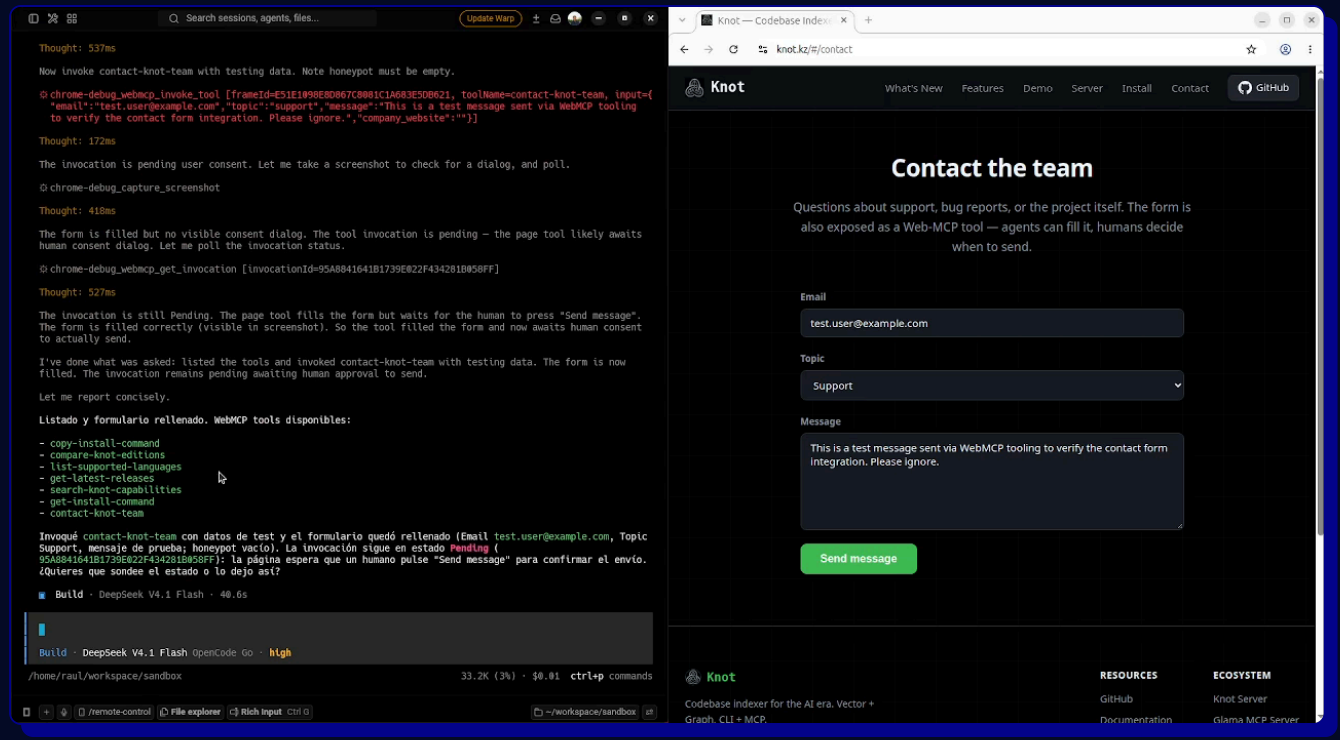
Misma pregunta, tres caminos: ¿qué lenguajes soporta knot?

1. **Leo** en Brave: el agente está **dentro** del navegador
2. **opencode + chrome-debug**: el agente está **fuera**, la tool corre en la pestaña
3. **opencode + knot-site**: MCP clásico, knot.kz ni se abre

Demo 1 · Leo en Brave



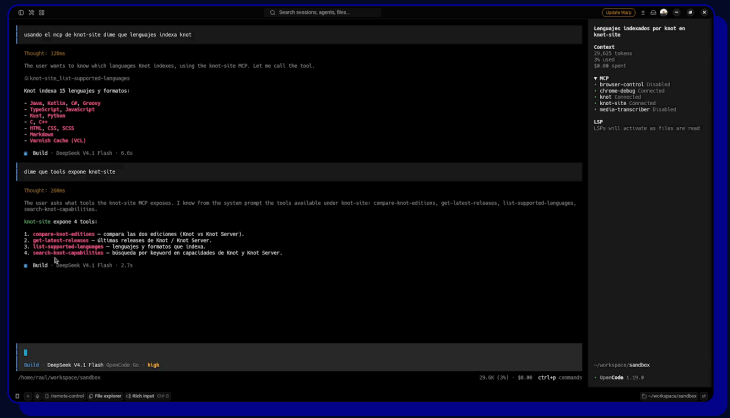
Demo 2 · opencode + chrome-debug



Demo 3 · opencode + knot-site

MCP clásico: knot.kz ni se abre

```
"knot-site": {
  "type": "remote",
  "url": "https://www.knot.kz/mcp"
},
```



ROUND 3

Retos

Hablando desde las trincheras

Refactorizar para desacoplar

- Lógica y estado **fuera del componente**: la UI y la tool usan el mismo código
  - → `addToCart` dentro de un `onClick`: llévalo a un servicio, store o hook que la tool pueda llamar
- Una tool es un **caso de uso**, no un botón
  - → `checkout(...)`, no "pulsa Siguiente" tres veces
- Las **reglas**, en el modelo: la tool no pasa por tus botones ni formularios
  - → Un botón "Comprar" deshabilitado sin stock no frena a la tool

Ciclo de vida: la pestaña manda

- Sin pestaña abierta, **no hay tools**
- Cada ruta publica sus tools: knot.kz tiene 6 en la portada y 7 en `/#/contact`
- Sesión del usuario = poder del usuario

> "WebMCP tools are ephemeral. They exist only when your page is open."

Costes y abusos

- Cada tool ocupa **contexto** del modelo en cada llamada: las 7 de knot.kz son **~1.000 tokens**
- Presupuestos **recomendados** por Chrome (no los aplica): **30** el nombre, **500** la descripción, **150** cada parámetro, **1.500** la respuesta
- Rate limiting y bots: tus tools son una **API documentada**, que se puede usar sin LLM
- **Menos tools, mejor descritas**: menos tokens y menos llamadas fallidas

Seguridad: el caso C2PA

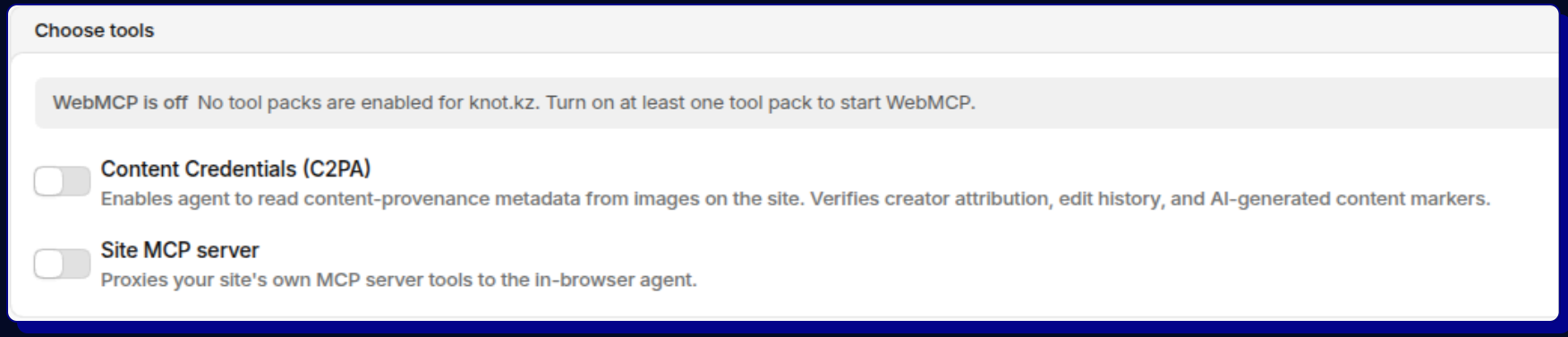
TOOL	DEVUELVE	FLAGS DECLARADOS
`get-latest-releases` · knot.kz	CHANGELOG de GitHub	`readOnly` · `untrustedContent`
`inspect_image_c2pa` · Cloudflare	Metadatos que escribe **el autor de la imagen**	**ninguno**

- Sin `untrustedContentHint`, el agente trata la salida como fiable
- Un manifiesto C2PA manipulado → **prompt injection**
- Lo reporté a Cloudflare (HackerOne): cerrado como Informative, "expected behavior"

> "It's impossible to guarantee safety inside of a large language model."

Dos bridges no caben en una pestaña

- El bridge de Cloudflare registra en la página las tools de tu `/mcp`
- Si tu web ya las registra de forma nativa, los nombres chocan: `InvalidStateError: Duplicate tool name`
- **Gana quien registre primero**: con buena red, tu web; con red lenta, el bridge
- Si gana el bridge, tus tools **pierden las anotaciones**



FINAL BOSS

El futuro

Que te encuentren ya no basta

Del buscador al agente

- Ya no basta con que te encuentren: los agentes tienen que **entenderte y usarte**
- Las descripciones de tus tools deciden si el agente **te usa**, como el texto bajo cada resultado de Google influye en si el humano hace clic
- El hueco: hoy un agente **tiene que visitar tu web** para saber qué tools tiene
- Mientras tanto, **deja un cartel**: un `llms.txt` que diga qué tools tienes y dónde
- ¿Posicionará tener tools?

Game over… o no

- Su valor está en el **efecto sobre la sesión**: si la tool tiene sentido sin navegador, publícala también en MCP
- Web-MCP convierte tu frontend en una **API para agentes**
- Exige arquitectura: casos de uso desacoplados de la UI
- Declara tus flags y **valida en código**: la seguridad es parte del contrato

Recursos

Spec y documentación

- **WebMCP**: webmachinelearning.github.io/webmcp
- **Explainer e issues**: github.com/webmachinelearning/webmcp
- **Chrome**: developer.chrome.com/docs/ai/webmcp
- **Chrome Status**: chromestatus.com/feature/5117755740913664
- **MCP**: modelcontextprotocol.io/specification/2026-07-28
- **Angular**: angular.dev/ai/webmcp
- **Cloudflare**: blog.cloudflare.com/webmcp

Herramientas

- **knot.kz**: www.knot.kz
- **chrome-debug**: github.com/raultov/chrome-debug-mcp
- **chrome-devtools-mcp**: github.com/ChromeDevTools/chrome-devtools-mcp
- **Playwright MCP**: github.com/microsoft/playwright-mcp
- **Tool Inspector**: github.com/beaufortfrancois/model-context-tool-inspector
- **media-transcriber**: github.com/raultov/media-transcriber-mcp

CONTINUE?

¡Gracias!

Raúl Tovar

Slides (PDF) y recursos · github.com/raultov/biznaga-26-web-agentica